

Core Language Working Group "tentatively ready" Issues for the June, 2018 (Rapperswil) meeting

Section references in this document reflect the section numbering of document [WG21 N4700](#).

2233. Function parameter packs following default arguments

Section: 11.3.6 [dcl.fct.default] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-02-25

The resolution of issue 777 attempts to make this valid:

```
template<typename ...T> void f(int n = 0, T ...t);
```

However, it fails to do so, since any parameters resulting from the expansion of the pack would be ordinary parameters without default arguments following a parameter with a default argument, which is ill-formed. Thus only an empty pack would be usable with such a declaration, which violates the restriction against such contexts in 17.7 [temp.res] bullet 8.3.

Proposed resolution, February, 2018:

1. Change 8.2.2 [expr.call] paragraph 4 as follows:

When a function is called, each parameter (11.3.5 [dcl.fct]) ~~shall be~~ **is** initialized (11.6 [dcl.init], 15.8 [class.copy], 15.1 [class.ctor]) with its corresponding argument. **If there is no corresponding argument, the default argument for the parameter is used; the program is ill-formed if one is not present. [Example:**

```
template<typename ...T> int f(int n = 0, T ...t);  
int x = f<int>();    // error: no argument for second function parameter
```

—end example] If the function is a non-static member function, the this parameter of the function (12.2.2.1 [class.this]) ~~shall be~~ **is** initialized with a pointer to the object of the call, converted as if by an explicit type conversion (8.4 [expr.cast]). [Note: There is no access or ambiguity checking...

2. Change 11.3.6 [dcl.fct.default] paragraph 1 as follows:

If an *initializer-clause* is specified in a *parameter-declaration* this *initializer-clause* is used as a default argument. **[Note:** Default arguments will be used in calls where trailing arguments are missing **(8.2.2 [expr.call]). —end note]**

3. Change 11.3.6 [dcl.fct.default] paragraph 4 as follows:

For non-template functions, default arguments can be added in later declarations of a function in the same scope. Declarations in different scopes have completely distinct sets of default arguments. That is, declarations in inner scopes do not acquire default arguments from declarations in outer scopes, and vice versa. In a given function declaration, each parameter subsequent to a parameter with a default argument shall have a default argument supplied in this or a previous declaration, **unless the parameter was expanded from a parameter pack**, or shall be a function parameter pack. A default argument shall not be redefined by a later declaration (not even to the same value). [Example:

```
void g(int = 0, ...);    // OK, ellipsis is not a parameter so it can follow
                        // a parameter with a default argument
void f(int, int);
void f(int, int = 7);
void h() {
    f(3);                // OK, calls f(3, 7)
    void f(int = 1, int); // error: does not use default from surrounding scope
}
void m() {
    void f(int, int);     // has no defaults
    f(4);                // error: wrong number of arguments
    void f(int, int = 5); // OK
    f(4); // OK, calls f(4, 5);
    void f(int, int = 5); // error: cannot redefine, even to same value
}
void n() {
    f(6);                // OK, calls f(6, 7)
}
template<class ... T> struct C {
    void f(int n = 0, T...);
};
C<int> c;                // OK; instantiates declaration void C::f(int n = 0, int)
```

2249. identifiers and id-expressions

Section: 8.1.4.1 [expr.prim.id.unqual] **Status:** tentatively ready **Submitter:** Robert Haberlach **Date:** 2016-03-17

According to 8.1.4.1 [expr.prim.id.unqual] paragraph 1,

An *identifier* is an *id-expression* provided it has been suitably declared (Clause 10 [dcl.dcl]).

Not only is an *identifier* an *id-expression* by (grammatical) definition, *declarator-id* is defined in terms of *id-expression*, which makes this circular. If the intention was to disallow use of undeclared identifiers as primary expressions, this should be altered accordingly.

Proposed resolution, February, 2018:

Change 8.1.4.1 [expr.prim.id.unqual] paragraph 1 as follows:

An *identifier* is **only** an *id-expression* **provided if** it has been suitably declared (Clause 10 [dcl.dcl]) **or if it appears as part of a declarator-id (Clause 11 [dcl.decl])**. [Note: For operator-function-ids, see...

2285. Issues with structured bindings

Section: 11.5 [dcl.struct.bind] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-07-01

1. What is the point of declaration of a name introduced by a structured binding? If it's the point at which it appears, we're missing a rule to make it ill-formed to reference the name before its type is deduced (similar to what we have for 'auto'). [Suggestion: point of declaration is after the *identifier-list*, program is ill-formed if the name is mentioned before the end of the initializer.]
2. Are structured bindings permitted at namespace scope? There doesn't seem to be a rule against that. If so, what's their linkage? Is it intentional that `static`, `extern` are disallowed? Should we only allow automatic storage duration? [Suggestion: only permit automatic storage duration, per the design paper.]
3. (If the answer to 2 is yes...) is the declaration in a variable template permitted to use structured bindings? If so, how do you name the result? (The bindings themselves aren't introduced as *template-names* by the current wording.) If not, we're missing a restriction on that. [Suggestion: no to question 2.]
4. Did we intend to guarantee that the object whose members are denoted by bindings is kept "together":

```
auto f() -> int (&)[2];
auto [x, y] = f();
assert(&x + 1 == &y); // ok?

struct S { int a, b, c; }; // standard-layout
auto [a,b,c] = S();
assert(&((S*)&a)->b == &b); // ok?
```

(If yes, this means we can't synthesize independent variables for each element of an array or struct that's bound in this way, and it's harder to remove dead components of a destructured object.) Obviously we may need to keep the object together if it has a non-trivial destructor. [Suggestion: do not allow reaching the complete object from a binding.]

5. Should the copy->move promotion be permitted for a return of a structured binding?

```
struct A { string s; int n; };
string f() {
    auto [s,n] = A();
    return s; // copy required here?
}
```

[Suggestion: allow promotion to move -- as if the binding were a real local variable -- if the implicit underlying variable is not a reference. Maybe also allow NRVO, depending on answer to question 8.]

Notes from the April, 2017 teleconference:

Items 1 and 3 are core issues; item 4 is NAD - the bindings are kept together, which is implied by the existing rules about copying the object into a hidden temporary. The remaining items are questions for EWG and new items in "extension" status will be opened for them.

Proposed resolution, February, 2018:

1. Change 6.3.2 [basic.scope.pdecl] paragraph 9 as follows and add the following new paragraph thereafter:

The point of declaration for a function-local predefined variable (11.4 [dcl.fct.def] 11.4.1 [dcl.fct.def.general]) is immediately before the *function-body* of a function definition.

The point of declaration of a structured binding (11.5 [dcl.struct.bind]) is immediately after the *identifier-list* of the structured binding declaration.

2. Add the following as a new paragraph following 11.5 [dcl.struct.bind] paragraph 1:

...taken from the corresponding structured binding declaration. The type of the *id-expression* *e* is called *E*. [Note: *E* is never a reference type (Clause 8 [expr]). —end note]

If the *initializer* refers to one of the names introduced by the structured binding declaration, the program is ill-formed.

(Note: In response to item 3, the current wording of 17 [temp] paragraph 1 does not allow templated structured binding declarations, and no change is being proposed.)

2351. void{}

Section: 8.2.3 [expr.type.conv] **Status:** tentatively ready **Submitter:** Peter Dimov **Date:** 2017-06-26

According to 8.2.3 [expr.type.conv] paragraph 2,

If the initializer is a parenthesized single expression, the type conversion expression is equivalent to the corresponding cast expression (8.4 [expr.cast]). Otherwise, if the type is *cv void* and the initializer is *()*, the expression is a prvalue of the specified type that performs no initialization. Otherwise, the expression is a prvalue of the specified type whose result object is direct-initialized (11.6 [dcl.init]) with the initializer. For an expression of the form $\tau()$, τ shall not be an array type.

It seems an oversight that `void{}` is not treated like `void()`.

Proposed resolution, April, 2018:

Change 8.2.3 [expr.type.conv] paragraph 2 as follows:

...Otherwise, if the type is *cv void* and the initializer is *()* or **`{}`**, the expression is a prvalue of the specified type that performs no initialization. Otherwise...

2356. Base class copy and move constructors should not be inherited

Section: 16.3.1 [over.match.funcs] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2018-02-26

Base class copy and move constructors brought into a derived class via a *using-declaration* should not be considered by overload resolution when constructing a derived class object.

Proposed resolution, February, 2018:

Change 16.3.1 [over.match.funcs] paragraph 8 as follows:

A defaulted move special function (15.8 [class.copy]) that is defined as deleted is excluded from the set of candidate functions in all contexts. **A constructor inherited from class type *C* (15.6.3 [class.inhctor.init]) that has a first parameter of type “reference to cv1 *P*” (including such a constructor instantiated from a template) is excluded from the set of candidate functions when constructing an object of type cv2 *D* if the argument list has exactly one argument and *C* is reference-related to *P* and *P* is reference-related to *D*. [Example:**

```
struct A {
    A();
    A(A &&); // #1
    template<typename T> A(T &&); // #2
};
struct B : A {
    using A::A;
    B(const B &); // #3
    B(B &&) = default; // #4, implicitly deleted

    struct X { X(X &&) = delete; } x;
};
extern B b1;
B b2 = static_cast<B&&>(b1); // calls #3: #1, #2, and #4 are not viable
struct C { operator B&&>(); };
B b3 = C(); // calls #3

—end example]
```

2359. Unintended copy initialization with designated initializers

Section: 11.6.1 [dcl.init.aggr] **Status:** tentatively ready **Submitter:** Zhihao Yuan **Date:** 2017-10-06

According to 11.6.1 [dcl.init.aggr] bullet 4.2,

Otherwise, the element is copy-initialized from the corresponding *initializer-clause* or the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*.

This sounds as if the initialization performed by a designated initializer is always copy-initialization. However, it was intended that the kind of initialization match the form of the initializer, i.e., a *designated-initializer-clause* of the form

```
{ .x{3} }
```

was intended to perform direct-initialization.

Proposed resolution, April, 2018:

Change 11.6.1 [dcl.init.aggr] bullet 4.2 as follows:

Otherwise, the element is copy-initialized from the corresponding *initializer-clause* or **is initialized with** the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*.

